

Advanced Audio Processing: Exercise 02

Audio classification with convolutional neural networks

Introduction

The goal of this lab exercise is to show you how to perform classification on audio data using convolutional neural networks (CNNs). You are asked to create an audio classification system that predicts one label for each input audio signal.

A typical task in modern audio signal processing is to get an audio signal and predict a class for it. Examples of such tasks are audio tagging [1], music genre recognition [2], and acoustic scene classification [3]. The usual approach is to extract some features from an audio signal, and give the extracted features as an input to an audio classification system.

In this exercise you will implement an audio classifier for multiclass data. In task 1 you will process data using the functions developed in the previous lab exercise (1.0 point). In task 2, you will design a DNN system based on CNNs to classify the input audio signal (1.0 point). And finally in task 3, you will do the training and testing of your system (1.0 point).

★ For this exercise you need to prepare and return the following files:

-
- `data_loader.py`
- `cnn_system.py`
- `cnn_training.py`

Please submit them along with the files from the previous exercise that are needed to run the code for this exercise (i.e., `file_io.py`, `getting_and_init_the_data.py`, and `dataset_class.py`).

★ Please place all files in the root folder named as

"E2_firstname_lastname_studentnumber" and upload it to the Moodle page as a .zip file.

★ Please check the required versions of Python and PyTorch from the Moodle front page "Details about exercises".

Task 1. Getting and initializing the data (1 point)

The data used for this exercise is from the GTZAN genre classification dataset, developed by G. Tzanetakis. The data is provided at the Moodle page of this lab exercise. The dataset has its faults [4], and should not be used for any real research, but works here for educational purposes. For the needs of this task, the subset is divided in three splits:

- Training split to develop your model.
- Validation split to optimize your model.
- Test split to measure the performance of your system.

For this task you have to follow below steps:

1. Create a directory to host the code and various files for the present exercise. This directory will be called root directory for the rest of this document.
2. Get the file “**E2_template.zip**” from the Moodle page and extract all files within that into the root directory.
3. Get the archive “**genres_dataset.zip**” from the Moodle page of the lab exercise, and expand it in the root directory. This will create the corresponding directory “genres_dataset”, containing genres.csv listing the class names, and the following three directories:
 - (a) training
 - (b) validation
 - (c) testing
4. You can find the functions to extract the features at the following files:
 - file_io.py
 - dataset_class.py
 - getting_and_init_the_data.py

Functions to load the data are in the file called **getting_and_init_the_data.py**, which makes use of the class MyDataset in the file **dataset_class.py**.

5. Next, you are going to set up three different data loaders, one for each group of features, i.e. training, validation, and testing. For that, open the file “**data_loader.py**” and implement the missing sections in the “load_data” function that are denoted by question marks.
6. A template for calling the “load_data” function on the “training” set is provided inside the main function. Repeat this for the “validation” and the “testing” splits by implementing the missing lines inside the “main” function. Note that for keeping track of the predictions of your system, the testing data loader should not shuffle the data.
7. Finally, to test your code, run the “main” function which should print the size of each split.

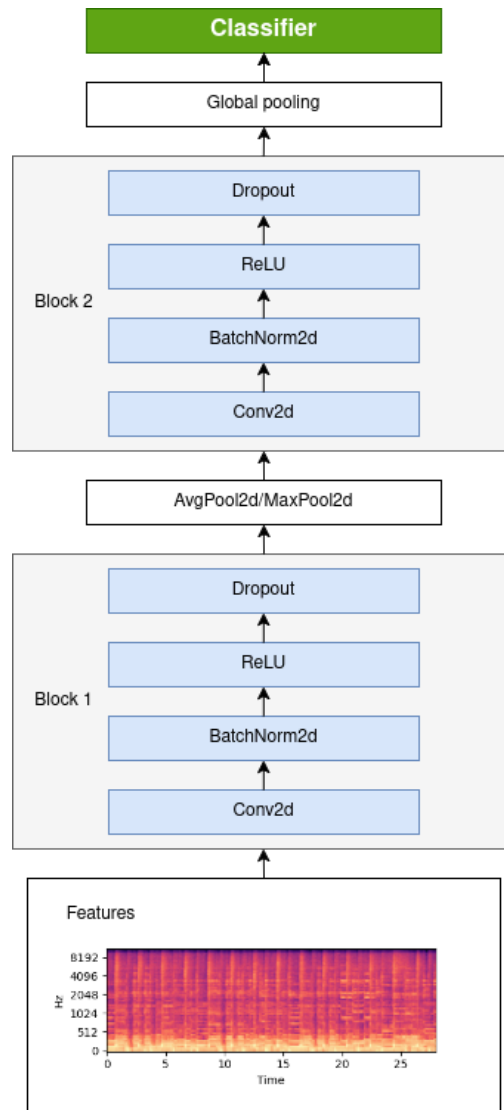


Figure 1. The CNN model to be implemented as an “audio classification” system (Task 2).

Task 2. Setting-up the CNN model (1 point)

In this task you will be focusing on implementing a CNN-based system. To do so, you will have to do the following:

1. Open the file “**cnn_system.py**” which includes a template for implementing your CNN model. Make the init method to set up two layers of CNNs (CNN block 1 and CNN block 2 in Figure 1), where each convolutional layer is followed by a rectified linear unit, a batch normalization process, and a max pooling process. Implement a helper function to create the CNN blocks. You should have the parameters for each layer to be defined using input argument to the `__init__` method. As an example you can see the provided code “**dnn_pytorch_example.py**”.
2. Add a dropout with a probability defined at the init method at the end of the CNN block_1.
3. After the second CNN block, the tensor dimensions corresponding to mel-bins and time frames should be pooled out. You can do this with `AdaptiveAvgPool2d` (or `AdaptiveMaxPool2d`). After the pooling layer, using `tensor.squeeze [5]` might be useful.

4. Finally, add a linear layer that will act as your classifier (output feature dimension is equal to the number of classes).
5. Implement the forward method, which will have as an input a PyTorch tensor, process it according to your CNN, and return the output of your linear layer.
6. Implement the missing lines inside the “main” function to build the CNN model.
7. Finally test your code by running the “main” function that is provided to test your class using random data.

Task 3. Training and testing (1 point)

After the set-up of a CNN-based system, you need to optimize its parameters. For that, different decisions have to be made. For example, how many CNN blocks (i.e. CNN, non-linearity, normalization, sub-sampling) to use, how many channels to use in each convolution layer, and what would be a good learning rate for the optimizer. For answering all such questions, the CNN-based system has to be trained with the training data and with a set of choices for the hyper-parameters, and then its performance has to be evaluated on the validation data. At the end, the best performing system is chosen and its performance is assessed using the testing data.

For this task you are free to experiment with the hyper-parameters, keeping those that give as result the best performance. To do so, you will have to implement the following:

1. Open the file “**cnn_training.py**” which includes a template for training your CNN system.
1. Define the optimizer as Adam and give it model parameters and a learning rate.
2. Define the loss function as cross entropy [6].
3. Implement the missing parts from training, validation and testing loops.
4. Finally, test your code by running the main function.
5. Experiment with different model parameters such as number of convolutional filters, and kernel sizes of convolutional and pooling layers. You can also try to add another block to the model. Report your observations as comments at the end of the “**cnn_training.py**” file. You can also implement the functionality to collect the confusion matrix in the testing loop, and inspect that to see which genres are getting mixed up.

Please note that the focus of the present exercise is not to get the best performing system. Instead, the focus is to correctly implement a typical audio classification scenario, where you will tune the hyper-parameters of your model.

References

- [1] <https://www.kaggle.com/c/freesound-audio-tagging-2019>
- [2] <https://www.crowdai.org/challenges/www-2018-challenge-learning-to-recognize-musical-genre>
- [3] <http://dcase.community/challenge2019/task-acoustic-scene-classification>
- [4] <https://arxiv.org/abs/1306.1461>
- [5] <https://docs.pytorch.org/docs/stable/generated/torch.squeeze.html>
- [6] <https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>